

## Parte 1:

1. La periodicidad del tono se representa con la ayuda de la función *np.concatenate*. Se agregan puntos de fantasma al principio y al final de la lista original de puntos de control, donde el primer punto es una copia del último restado con el tamaño del círculo, y viceversa con el último punto. Esto permite que la función de interpolación lineal *np.interp* calcule correctamente los valores cuando un píxel cae entre el final y el inicio del espectro, evitando saltos bruscos o discontinuidades alrededor del color rojo.

2. La función *np.interp* determina los puntos a usar buscando en lista cuáles son los dos valores exactos que encierran o rodean al tono de un píxel. Los pesos de interpolación se calculan basándose en la distancia a la que se encuentra el píxel de cada uno de estos extremos.

3. La función *g\_m* se define como una multiplicación:  $S' = S * m(h)$  para el modo *hsv*, y  $c' = c * m(h^*)$  para el modo *lch*. El valor neutro del parámetro *m* es 1, ya que al multiplicar se mantiene exactamente igual sin sufrir cambios. Al acercarse o superar los extremos del rango permitido, se aplica la función *np.clip* para dejarlo en el tope máximo válido.

4. Se realiza en el primer bloque condicional *if mode == 'HSV'* y *elif mode == 'LCH'*. En el modo HSV, las componentes que permanecen sin modificar son el canal del H y V. Mientras que, en el LCH las componentes sin cambiar son el canal L y h.

5. En el modo HSV, la nueva saturación se recorta al rango entre 0 y 1, mientras que en el modo *lch*, el croma se limita al rango de 0 a infinito positivo. Finalmente, al volver la imagen a RGB, se vuelve a aplicar *np.clip* entre 0 y 1 a todos los canales.

6. *pixel.py*

7. Un mismo mapeo no da el mismo resultado porque cada espacio de color representa la luz y los colores de manera distinta. El espacio *lch* está diseñado para imitar cómo funciona la vista humana. Este espacio de color separa la luz del color por completo. Mientras que, el espacio *hsv* no está diseñado en base a cómo ven nuestros ojos, por lo que al aumentar la saturación este indirectamente aumenta el brillo debido a que este modelo mezcla la intensidad del color con el brillo.

## Parte 2:

1. Las regiones que contribuyen a determinar el nuevo valor de salida son todas las que contienen dentro de su área al píxel determinado. Estas regiones se seleccionan a partir de la creación de una cuadrícula de coordenadas las cuales se calculan usando saltos definidos por el parámetro *step\_size*.
2. La CDF se construye calculando primero el histograma de la región con *np.histogram*, aplicando la limitación de contraste si es necesario, y luego sumando los valores acumulativamente mediante *hist.cumsum()*. Finalmente, se normaliza para que sus valores vayan de 0 a 255.
3. Si el número de bins es menor que los niveles de intensidad, el histograma agruparía niveles distintos dentro del mismo grupo. Como resultado, píxeles que tenían pequeñas diferencias de tono recibirán exactamente el mismo valor de salida de la CDF. Esto destruiría los gradientes suaves.
4. Cada bloque se multiplica por una matriz de pesos antes de sumarse al resultado final. Esta matriz les da la máxima influencia a los píxeles del centro de la región y va disminuyendo ese valor gradualmente hacia los bordes. Mientras mayor sea el overlap, más regiones se superponen para calcular ese promedio conjunto, logrando una transición mucho más fluida y sin cortes visibles.
5. El parámetro *clip\_limit* actúa justo después de calcular el histograma y antes de la CDF, recortando los picos excesivos y repartiendo ese sobrante. Si el *clip\_limit* es muy alto, entonces es posible que no recorte nada. Sube el contraste al máximo, pero amplifica muchísimo el ruido en zonas lisas. Mientras que, si tiene un valor muy bajo, se aplanan el histograma casi por completo, por lo que la imagen no sufre casi cambios de contraste y se queda exactamente igual.
6. Al pasar a una única región que cubre toda la imagen, la cuadrícula de coordenadas no puede deslizarse. Entonces, se procesa la foto completa como un solo bloque. Como no hay otras regiones con las cuales solaparse, se cancela el efecto de la división por pesos y el algoritmo se comporta idénticamente a una ecualización global clásica.
7. Los bordes de la imagen se manejan limitando las coordenadas mediante la función *np.arange*, asegurando que la ventana de procesamiento no intente leer fuera de la imagen. Solo se procesan regiones que caben completamente dentro del marco, deteniéndose en el último bloque válido sin realizar padding.
8. En una región casi homogénea, la mayoría de los píxeles tienen el mismo tono, creando un peak masivo en el histograma. Esto genera una CDF que sube de forma casi vertical. Debido a esta pendiente extrema, cambios mínimos causados por ruido se transformarán en diferencias de color muy grandes en la salida, amplificando visualmente el ruido.

9. El costo computacional está dominado por la iteración de la malla y las multiplicaciones de la matriz de pesos con la región. Al disminuir el `step_size` para aumentar el `overlap`, se generan muchas más regiones, disparando el tiempo de ejecución. Al aumentar el tamaño de las regiones, las matrices temporales crecen, exigiendo más memoria y haciendo cada paso más lento.

### Parte 3:

1. Se realiza mediante un mapeo inverso ajustado al centro de los píxeles. A las coordenadas de la malla de salida se les suma 0.5, se dividen por el factor de escala `s` y luego se les resta 0.5.

2. Si el cálculo de las dimensiones de la nueva imagen genera números con decimales, la función hace `int(np.round(H * s))`, asegurando así que la matriz de salida tenga dimensiones válidas.

3. Para seleccionar los cuatro vecinos en el modo bilineal, se calcula el vecino superior izquierdo aplicando `np.floor` para obtener la parte entera de las coordenadas originales (`i0`, `j0`). Los otros tres vecinos se obtienen sumando 1 a dichas coordenadas (`i1 = i0 + 1`, `j1 = j0 + 1`).

4. Se calculan según la porción decimal de las coordenadas (las distancias `dy` y `dx` hacia el vecino superior izquierdo). Su suma es 1 porque distribuyen el 100% de la influencia del color dentro del área formada por los cuatro vecinos.

5. Cuando la posición original no tiene decimales, las distancias `dx` y `dy` son 0. Esto provoca que el peso valga 1, mientras que los pesos `w01`, `w10` y `w11` valen 0, copiando directamente el valor de ese píxel sin mezclar colores de los demás vecinos.

6. Se maneja con la función `np.clip`. Se utiliza tanto para limitar las coordenadas mapeadas `u` y `v` al rango de la imagen original, como para asegurar que al sumar 1 para buscar a los vecinos, no se exceda el índice máximo (`H - 1`), repitiendo así el píxel del borde.

8. Tienen en común el cálculo de dimensiones de salida, la creación de la malla de coordenadas y la fórmula de mapeo inverso. Difieren en que el modo 'vecino' simplemente redondea las coordenadas inversas para copiar un único píxel, mientras que 'bilineal' separa la parte entera y decimal para ponderar cuatro píxeles.

9. Porque se acumulan múltiples operaciones de redondeo o interpolaciones de color. Esto genera una pérdida de información en cada paso, causando un desenfoque progresivo en la imagen resultante.

